

Module 5: Pandas DataFrame

Author: Lei Wu

Date: 2023-01-01

Version: 1.0.0

Learning Objectives

- Understanding DataFrame data structure
- DataFrame creation and manipulation
- Indexing, selection, and filtering
- Data cleaning and preprocessing
- Basic data analysis operations

Topics Covered

1. Introduction to DataFrames
2. Creating DataFrames from various sources
3. DataFrame attributes and methods
4. Indexing and selection (loc, iloc, at, iat)
5. Filtering and boolean indexing
6. Adding and removing columns/rows
7. Handling missing values
8. Data type conversion
9. Sorting and ranking
10. Basic aggregation functions
11. DataFrame operations and arithmetic

Exercises

- DataFrame creation and manipulation exercises
- Data cleaning and preprocessing tasks

1. Introduction to DataFrames

In [...

```
1  import pandas as pd
2  import numpy as np
3
4
5  # Create a sample DataFrame
6  data = {
7      'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
8      'Age': [25, 30, 35, 28, 32],
9      'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Ph
```

```

10         'Salary': [50000, 60000, 70000, 55000, 65000]
11     }
12
13     df = pd.DataFrame(data)
14     print("Sample DataFrame:")
15     print(df)
16
17     # DataFrame properties
18     print(f"\nDataFrame Properties:")
19     print(f"Shape: {df.shape}")
20     print(f"Columns: {df.columns.tolist()}")
21     print(f"Index: {df.index.tolist()}")
22     print(f>Data types:\n{df.dtypes}")
23     print(f"Memory usage: {df.memory_usage().sum()} bytes")
24
25     # Basic information
26     print(f"\nDataFrame Info:")
27     print(df.info())
28
29     # Statistical summary
30     print(f"\nStatistical Summary:")
31     print(df.describe())

```

3. DataFrame Indexing and Selection

In [...

```

1     # Create a comprehensive DataFrame for indexing examples
2     data = {
3         'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank',
4         'Age': [25, 30, 35, 28, 32, 45, 29, 38],
5         'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Ph
6         'Salary': [50000, 60000, 70000, 55000, 65000, 80000, 58000, 7
7         'Department': ['IT', 'HR', 'IT', 'Finance', 'IT', 'Finance',
8     }
9
10    df = pd.DataFrame(data, index=['A', 'B', 'C', 'D', 'E', 'F', 'G',
11
12    print(f"Original DataFrame:\n{df}")
13
14    # Column selection
15    print(f"\n--- Column Selection ---")
16    print(f"Single column (Name):\n{df['Name']}")
17    print(f"Multiple columns:\n{df[['Name', 'Age', 'Salary']]}")
18
19    # Row selection using iloc (position-based)
20    print(f"\n--- Row Selection (iloc) ---")
21    print(f"First row:\n{df.iloc[0]}")
22    print(f"First 3 rows:\n{df.iloc[:3]}")
23    print(f"Rows 2 to 5:\n{df.iloc[2:6]}")
24    print(f>Last 2 rows:\n{df.iloc[-2:]}")
25
26    # Row and column selection using iloc
27    print(f"\n--- Row and Column Selection (iloc) ---")
28    print(f"First 3 rows, first 3 columns:\n{df.iloc[:3, :3]}")
29    print(f"Rows 1, 3, 5 and columns 0, 2, 4:\n{df.iloc[[1, 3, 5], [0
30
31    # Label-based selection using loc
32    print(f"\n--- Label-based Selection (loc) ---")
33

```

```

34     print(f"Row with index 'C':\n{df.loc['C']}")
35     print(f"Rows 'A' to 'D':\n{df.loc['A':'D']}")
36     print(f"Rows 'B', 'D', 'F' and columns 'Name', 'Salary':\n{df.loc
37
38     # Boolean indexing
39     print(f"\n--- Boolean Indexing ---")
40     print(f"People with age > 30:\n{df[df['Age'] > 30]}")
41     print(f"People in IT department:\n{df[df['Department'] == 'IT']}")
42     print(f"People with salary between 50000 and 70000:\n{df[(df['Sal
43
44     # Using .at and .iat for single value access
45     print(f"\n--- Single Value Access ---")
46     print(f"Value at row 0, column 'Name': {df.at['A', 'Name']}")
47     print(f"Value at position (2, 3): {df.iat[2, 3]}")
48
49     # Setting values
50     df_copy = df.copy()
51     df_copy.loc['A', 'Salary'] = 55000
52     df_copy.iloc[1, 2] = 'San Francisco'
    print(f"\nAfter modification:\n{df_copy}")

```

4. Data Cleaning and Preprocessing

In [...

```

1     # Create a DataFrame with various data quality issues
2     data_messy = {
3         'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank',
4         'Age': [25, 30, 35, 28, 32, 45, 29, 38, np.nan, 42],
5         'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Ph
6         'Salary': [50000, 60000, 70000, 55000, 65000, 80000, 58000, 7
7         'Department': ['IT', 'HR', 'IT', 'Finance', 'IT', 'Finance',
8         'Email': ['alice@company.com', 'bob@company.com', 'charlie@co
9         'eve@company.com', 'frank@company.com', 'grace@comp
10        'ivy@company.com', 'jack@company.com'],
11        'Phone': ['555-1234', '555-5678', '555-9012', '555-3456', '55
12        '555-2345', '555-6789', '555-0123', '555-4567', '55
13
14    }
15
16    # Add some missing values and inconsistencies
17    df_messy = pd.DataFrame(data_messy)
18    df_messy.loc[2, 'City'] = 'chicago' # Inconsistent case
19    df_messy.loc[5, 'Department'] = 'FINANCE' # Inconsistent case
20    df_messy.loc[7, 'Email'] = np.nan # Missing email
21    df_messy.loc[8, 'Phone'] = '555-XXXX' # Invalid phone
22
23    print(f"Messy DataFrame:\n{df_messy}")
24
25    # Check for missing values
26    print(f"\n--- Missing Values Analysis ---")
27    print(f"Missing values per column:\n{df_messy.isnull().sum()}")
28    print(f"Percentage of missing values:\n{(df_messy.isnull().sum())
29
30    # Check for duplicates
31    print(f"\n--- Duplicate Analysis ---")
32    print(f"Number of duplicate rows: {df_messy.duplicated().sum()}")
33    print(f"Duplicate rows:\n{df_messy[df_messy.duplicated()]}")
34
35

```

```

36 # Data type analysis
37 print(f"\n--- Data Type Analysis ---")
38 print(f"Data types:\n{df_messy.dtypes}")
39
40 # Handle missing values
41 print(f"\n--- Handling Missing Values ---")
42 # Fill missing age with median
43 df_clean = df_messy.copy()
44 df_clean['Age'] = df_clean['Age'].fillna(df_clean['Age'].median())
45 print(f"After filling missing age with median:\n{df_clean['Age']}")
46
47 # Fill missing email with 'unknown@company.com'
48 df_clean['Email'] = df_clean['Email'].fillna('unknown@company.com')
49 print(f"After filling missing email:\n{df_clean['Email']}")
50
51 # Standardize text data
52 print(f"\n--- Text Standardization ---")
53 df_clean['City'] = df_clean['City'].str.title()
54 df_clean['Department'] = df_clean['Department'].str.title()
55 print(f"Standardized City:\n{df_clean['City']}")
56 print(f"Standardized Department:\n{df_clean['Department']}")
57
58 # Data validation
59 print(f"\n--- Data Validation ---")
60 # Check email format
61 email_pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
62 valid_emails = df_clean['Email'].str.match(email_pattern)
63 print(f"Valid email addresses: {valid_emails.sum()}/{len(df_clean['Email'])}")
64
65 # Check phone format
66 phone_pattern = r'^\d{3}-\d{4}$'
67 valid_phones = df_clean['Phone'].str.match(phone_pattern)
68 print(f"Valid phone numbers: {valid_phones.sum()}/{len(df_clean['Phone'])}")
69
70 # Create age groups
71 print(f"\n--- Feature Engineering ---")
72 df_clean['Age_Group'] = pd.cut(df_clean['Age'],
73                                bins=[0, 30, 40, 50, 100],
74                                labels=['Young', 'Middle', 'Senior'])
75 print(f"Age groups:\n{df_clean['Age_Group'].value_counts()}")
76
77 # Create salary categories
78 df_clean['Salary_Category'] = pd.cut(df_clean['Salary'],
79                                      bins=[0, 55000, 65000, 75000, 100000],
80                                      labels=['Low', 'Medium', 'High'])
81 print(f"Salary categories:\n{df_clean['Salary_Category'].value_counts()}")
82
83 print(f"\nCleaned DataFrame:\n{df_clean}")

```

2. Creating DataFrames from Various Sources

In [...

```

1 # 1. From dictionary
2 data_dict = {
3     'A': [1, 2, 3, 4],
4     'B': [5, 6, 7, 8],
5     'C': [9, 10, 11, 12]

```

```

6     }
7     df1 = pd.DataFrame(data_dict)
8     print("From dictionary:")
9     print(df1)
10
11    # 2. From list of lists
12    data_list = [[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
13    df2 = pd.DataFrame(data_list, columns=['A', 'B', 'C'])
14    print("\nFrom list of lists:")
15    print(df2)
16
17    # 3. From NumPy array
18    np_array = np.random.rand(4, 3)
19    df3 = pd.DataFrame(np_array, columns=['X', 'Y', 'Z'])
20    print("\nFrom NumPy array:")
21    print(df3)
22
23    # 4. From list of dictionaries
24    data_dicts = [
25        {'Name': 'Alice', 'Age': 25, 'City': 'NYC'},
26        {'Name': 'Bob', 'Age': 30, 'City': 'LA'},
27        {'Name': 'Charlie', 'Age': 35, 'City': 'Chicago'}
28    ]
29    df4 = pd.DataFrame(data_dicts)
30    print("\nFrom list of dictionaries:")
31    print(df4)
32
33    # 5. From CSV-like string
34    csv_string = """Name,Age,City
35    Alice,25,New York
36    Bob,30,Los Angeles
37    Charlie,35,Chicago"""
38    from io import StringIO
39    df5 = pd.read_csv(StringIO(csv_string))
40    print("\nFrom CSV string:")
41    print(df5)
42
43    # 6. Empty DataFrame with specified columns
44    df6 = pd.DataFrame(columns=['Name', 'Age', 'City'])
45    print("\nEmpty DataFrame:")
46    print(df6)
47

```

3. DataFrame Attributes and Methods

In [...

```

1     # Create a comprehensive DataFrame for demonstrating attributes a
2     data = {
3         'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank',
4         'Age': [25, 30, 35, 28, 32, 45, 29, 38],
5         'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Ph
6         'Salary': [50000, 60000, 70000, 55000, 65000, 80000, 58000, 7
7         'Department': ['IT', 'HR', 'IT', 'Finance', 'IT', 'Finance',
8         'Experience': [2, 5, 8, 3, 6, 12, 4, 9]
9     }
10    df = pd.DataFrame(data, index=['A', 'B', 'C', 'D', 'E', 'F', 'G',
11    print("Original DataFrame:")
12
13

```

```

14     print(df)
15
16     # Basic attributes
17     print(f"\n--- Basic Attributes ---")
18     print(f"Shape: {df.shape}")
19     print(f"Size: {df.size}")
20     print(f"Dimensions: {df.ndim}")
21     print(f"Index: {df.index}")
22     print(f"Columns: {df.columns}")
23     print(f>Data types:\n{df.dtypes}")
24     print(f"Memory usage:\n{df.memory_usage()}")
25
26     # DataFrame information
27     print(f"\n--- DataFrame Information ---")
28     print("Info method:")
29     print(df.info())
30
31     # Statistical methods
32     print(f"\n--- Statistical Methods ---")
33     print("Describe method (numeric columns only):")
34     print(df.describe())
35
36     print("\nDescribe method (include all columns):")
37     print(df.describe(include='all'))
38
39     # Value counts for categorical data
40     print(f"\n--- Value Counts ---")
41     print("Department value counts:")
42     print(df['Department'].value_counts())
43
44     print("\nCity value counts:")
45     print(df['City'].value_counts())
46
47     # Head and tail methods
48     print(f"\n--- Head and Tail ---")
49     print("First 3 rows:")
50     print(df.head(3))
51
52     print("\nLast 3 rows:")
53     print(df.tail(3))
54
55     # Sample method
56     print(f"\n--- Sampling ---")
57     print("Random sample of 3 rows:")
58     print(df.sample(3))
59
60     print("\nRandom sample of 50% of data:")
61     print(df.sample(frac=0.5))
62
63     # Transpose
64     print(f"\n--- Transpose ---")
65     print("Transposed DataFrame:")
66     print(df.T)
67
68     # Copy method
69     print(f"\n--- Copy Method ---")
70     df_copy = df.copy()
71     df_copy.loc['A', 'Salary'] = 99999
72     print("Original DataFrame (unchanged):")
73     print(df['Salary'].head(3))

```

```

78     print("\nCopied DataFrame (modified):")
79     print(df_copy['Salary'].head(3))
80
81     # Check for empty DataFrame
82     print(f"\n--- Empty Check ---")
83     print(f"Is DataFrame empty? {df.empty}")
84     print(f"Number of rows: {len(df)}")
85
86     # Check for null values
87     print(f"\n--- Null Value Check ---")
88     print("Null values per column:")
89     print(df.isnull().sum())
90     print(f"\nAny null values? {df.isnull().any().any()}")
91
92     # Check for duplicates
93     print(f"\n--- Duplicate Check ---")
94     print(f"Number of duplicate rows: {df.duplicated().sum()}")
95     print(f"Any duplicates? {df.duplicated().any()}")
96
97     # Column operations
98     print(f"\n--- Column Operations ---")
99     print("Column names as list:")
100    print(df.columns.tolist())
101
102    print("\nColumn data types:")
103    print(df.dtypes.to_dict())
104
105    # Index operations
106    print(f"\n--- Index Operations ---")
107    print("Index as list:")
108    print(df.index.tolist())
109
110    print("\nReset index:")
111    df_reset = df.reset_index()
112    print(df_reset.head(3))
113
114    print("\nSet index to Name:")
115    df_name_index = df.set_index('Name')
116    print(df_name_index.head(3))

```

5. Filtering and Boolean Indexing

In [...

```

1     # Create a comprehensive DataFrame for filtering examples
2     data = {
3         'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank',
4         'Age': [25, 30, 35, 28, 32, 45, 29, 38, 26, 42],
5         'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Ph
6         'Salary': [50000, 60000, 70000, 55000, 65000, 80000, 58000, 7
7         'Department': ['IT', 'HR', 'IT', 'Finance', 'IT', 'Finance',
8         'Experience': [2, 5, 8, 3, 6, 12, 4, 9, 1, 10],
9         'Performance': ['Good', 'Excellent', 'Good', 'Average', 'Exce
10    }
11
12    df = pd.DataFrame(data, index=['A', 'B', 'C', 'D', 'E', 'F', 'G',
13
14    print("Original DataFrame:")
15    print(df)

```

```

16
17 # Basic boolean indexing
18 print(f"\n--- Basic Boolean Indexing ---")
19 print("People with age > 30:")
20 print(df[df['Age'] > 30])
21
22 print("\nPeople in IT department:")
23 print(df[df['Department'] == 'IT'])
24
25 print("\nPeople with salary >= 60000:")
26 print(df[df['Salary'] >= 60000])
27
28 # Multiple conditions with logical operators
29 print(f"\n--- Multiple Conditions ---")
30 print("IT employees with salary > 60000:")
31 it_high_salary = df[(df['Department'] == 'IT') & (df['Salary'] >
32 print(it_high_salary)
33
34 print("\nPeople aged 25-35 OR with salary > 70000:")
35 age_salary_filter = df[(df['Age'] >= 25) & (df['Age'] <= 35) | (d
36 print(age_salary_filter)
37
38 print("\nPeople in IT OR Finance departments:")
39 dept_filter = df[(df['Department'] == 'IT') | (df['Department'] =
40 print(dept_filter)
41
42 # Using isin() for multiple values
43 print(f"\n--- Using isin() Method ---")
44 print("People in IT or HR departments:")
45 print(df[df['Department'].isin(['IT', 'HR'])])
46
47 print("\nPeople in major cities:")
48 major_cities = ['New York', 'Los Angeles', 'Chicago', 'Houston']
49 print(df[df['City'].isin(major_cities)])
50
51 # String filtering
52 print(f"\n--- String Filtering ---")
53 print("Names starting with 'A':")
54 print(df[df['Name'].str.startswith('A')])
55
56 print("\nNames containing 'ar':")
57 print(df[df['Name'].str.contains('ar', case=False)])
58
59 print("\nNames ending with 'e':")
60 print(df[df['Name'].str.endswith('e')])
61
62 # Numeric range filtering
63 print(f"\n--- Numeric Range Filtering ---")
64 print("Age between 30 and 40 (inclusive):")
65 print(df[(df['Age'] >= 30) & (df['Age'] <= 40)])
66
67 print("\nSalary in top 25% (above 75th percentile):")
68 salary_75th = df['Salary'].quantile(0.75)
69 print(f"75th percentile salary: {salary_75th}")
70 print(df[df['Salary'] > salary_75th])
71
72 # Using query() method
73 print(f"\n--- Using query() Method ---")
74 print("IT employees with experience > 5 years:")
75 print(df.query("Department == 'IT' and Experience > 5"))
76

```



```

80
81 print("\nEmployees aged 25-35 with salary 50000-70000:")
82 print(df.query("Age >= 25 and Age <= 35 and Salary >= 50000 and S
83
84 # Negation with ~ operator
85 print(f"\n--- Negation with ~ Operator ---")
86 print("People NOT in IT department:")
87 print(df[~df['Department'].isin(['IT'])])
88
89 print("\nPeople with salary NOT between 55000 and 65000:")
90 print(df[~((df['Salary'] >= 55000) & (df['Salary'] <= 65000))])
91
92 # Complex filtering with multiple criteria
93 print(f"\n--- Complex Filtering ---")
94 print("Senior employees (age > 35) in IT with excellent performan
95 senior_it_excellent = df[
96     (df['Age'] > 35) &
97     (df['Department'] == 'IT') &
98     (df['Performance'] == 'Excellent')
99 ]
100
101 print(senior_it_excellent)
102
103 print("\nHigh performers (Excellent or Good) with salary > 60000:
104 high_performers = df[
105     (df['Performance'].isin(['Excellent', 'Good'])) &
106     (df['Salary'] > 60000)
107 ]
108
109 print(high_performers)
110
111 # Filtering with null values
112 print(f"\n--- Filtering with Null Values ---")
113 # Add some null values for demonstration
114 df_with_nulls = df.copy()
115 df_with_nulls.loc['A', 'Experience'] = np.nan
116 df_with_nulls.loc['C', 'Performance'] = np.nan
117
118 print("DataFrame with some null values:")
119 print(df_with_nulls[['Name', 'Experience', 'Performance']])
120
121 print("\nRows with non-null Experience:")
122 print(df_with_nulls[df_with_nulls['Experience'].notna()])
123
124 print("\nRows with null Performance:")
125 print(df_with_nulls[df_with_nulls['Performance'].isna()])
126
127 # Using where() method
128 print(f"\n--- Using where() Method ---")
129 print("Salary where age > 30, otherwise NaN:")
130 print(df.where(df['Age'] > 30)['Salary'])
131
132 # Using mask() method (opposite of where)
133 print(f"\n--- Using mask() Method ---")
134 print("Salary where age <= 30, otherwise NaN:")
135 print(df.mask(df['Age'] > 30)['Salary'])

```

6. Adding and Removing Columns/Rows

In [...

```
1  # Create a base DataFrame for demonstrating column and row operat
2  data = {
3      'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
4      'Age': [25, 30, 35, 28, 32],
5      'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Ph
6      'Salary': [50000, 60000, 70000, 55000, 65000]
7  }
8
9  df = pd.DataFrame(data, index=['A', 'B', 'C', 'D', 'E'])
10
11  print("Original DataFrame:")
12  print(df)
13
14  # Adding columns
15  print(f"\n--- Adding Columns ---")
16
17  # Method 1: Direct assignment
18  df['Department'] = ['IT', 'HR', 'IT', 'Finance', 'IT']
19  print("After adding Department column:")
20  print(df)
21
22  # Method 2: Using assign() method
23  df = df.assign(Experience=[2, 5, 8, 3, 6])
24  print("\nAfter adding Experience column using assign():")
25  print(df)
26
27  # Method 3: Adding multiple columns at once
28  df[['Performance', 'Manager']] = [['Good', 'Excellent', 'Good', '
29                                     ['John', 'Jane', 'John', 'Mike']
30
31  print("\nAfter adding Performance and Manager columns:")
32  print(df)
33
34  # Method 4: Adding calculated columns
35  df['Salary_Plus_Bonus'] = df['Salary'] * 1.1 # 10% bonus
36  df['Age_Group'] = df['Age'].apply(lambda x: 'Young' if x < 30 els
37
38  print("\nAfter adding calculated columns:")
39  print(df)
40
41  # Method 5: Adding column with insert()
42  df.insert(2, 'Employee_ID', ['E001', 'E002', 'E003', 'E004', 'E00
43  print("\nAfter inserting Employee_ID at position 2:")
44  print(df)
45
46  # Removing columns
47  print(f"\n--- Removing Columns ---")
48
49  # Method 1: Using drop() with columns parameter
50  df_drop_cols = df.drop(columns=['Manager', 'Age_Group'])
51  print("After dropping Manager and Age_Group columns:")
52  print(df_drop_cols)
53
54  # Method 2: Using del statement
55  df_del = df.copy()
56  del df_del['Performance']
57  print("\nAfter deleting Performance column using del:")
58  print(df_del)
59
60  # Method 3: Using pop() method (returns the column)
61  df_pop = df.copy()
```

```

63 popped_column = df_pop.pop('Employee_ID')
64 print(f"\nPopped column (Employee_ID):\n{popped_column}")
65 print("\nDataFrame after popping Employee_ID:")
66 print(df_pop)
67
68 # Adding rows
69 print(f"\n--- Adding Rows ---")
70
71 # Method 1: Using loc with new index
72 df.loc['F'] = ['Frank', 45, 'Miami', 80000, 'Finance', 12, 'Excel']
73 print("After adding Frank's row using loc:")
74 print(df)
75
76 # Method 2: Using append() method
77 new_row = pd.Series(['Grace', 29, 'Seattle', 58000, 'HR', 4, 'Good'],
78                     index=df.columns, name='G')
79 df = df.append(new_row)
80 print("\nAfter adding Grace's row using append:")
81 print(df)
82
83 # Method 3: Using concat() method
84 new_rows = pd.DataFrame({
85     'Name': ['Henry', 'Ivy'],
86     'Age': [38, 26],
87     'City': ['Boston', 'Denver'],
88     'Salary': [72000, 62000],
89     'Department': ['IT', 'Marketing'],
90     'Experience': [9, 1],
91     'Performance': ['Excellent', 'Good'],
92     'Manager': ['John', 'Lisa'],
93     'Salary_Plus_Bonus': [79200, 68200],
94     'Age_Group': ['Senior', 'Young']
95 }, index=['H', 'I'])
96 df = pd.concat([df, new_rows])
97 print("\nAfter adding Henry and Ivy using concat:")
98 print(df)
99
100 # Removing rows
101 print(f"\n--- Removing Rows ---")
102
103 # Method 1: Using drop() with index
104 df_drop_rows = df.drop(['A', 'C'])
105 print("After dropping rows A and C:")
106 print(df_drop_rows)
107
108 # Method 2: Using drop() with conditions
109 df_condition_drop = df.drop(df[df['Age'] < 30].index)
110 print("\nAfter dropping rows where Age < 30:")
111 print(df_condition_drop)
112
113 # Method 3: Using boolean indexing to keep rows
114 df_keep_rows = df[df['Department'] != 'Marketing']
115 print("\nAfter keeping only non-Marketing employees:")
116 print(df_keep_rows)
117
118 # Method 4: Using dropna() to remove rows with missing values
119 df_with_nulls = df.copy()
120 df_with_nulls.loc['A', 'Salary'] = np.nan
121 df_with_nulls.loc['C', 'Department'] = np.nan
122 print("\nDataFrame with some null values:")

```

```

127 print(df_with_nulls[['Name', 'Salary', 'Department']])
128
129 df_no_nulls = df_with_nulls.dropna()
130 print("\nAfter removing rows with any null values:")
131 print(df_no_nulls[['Name', 'Salary', 'Department']])
132
133 # Method 5: Using dropna() with specific columns
134 df_drop_specific_nulls = df_with_nulls.dropna(subset=['Salary'])
135 print("\nAfter removing rows with null Salary:")
136 print(df_drop_specific_nulls[['Name', 'Salary', 'Department']])
137
138 # Reindexing
139 print(f"\n--- Reindexing ---")
140
141 # Reset index
142 df_reset = df.reset_index()
143 print("After resetting index:")
144 print(df_reset.head())
145
146 # Set new index
147 df_new_index = df.set_index('Name')
148 print("\nAfter setting Name as index:")
149 print(df_new_index.head())
150
151 # Reorder columns
152 print(f"\n--- Reordering Columns ---")
153 df_reordered = df[['Name', 'Age', 'Department', 'Salary', 'City',
154 print("After reordering columns:")
155 print(df_reordered.head())
156
157 # Using reindex() for columns
158 df_reindex_cols = df.reindex(columns=['Name', 'Salary', 'Age', 'D
159 print("\nAfter reindexing columns:")
160 print(df_reindex_cols.head())
161
162 # Using reindex() for rows
163 df_reindex_rows = df.reindex(['A', 'B', 'C', 'F', 'G', 'H', 'I',
164 print("\nAfter reindexing rows (with some missing):")
165 print(df_reindex_rows.head())
166
167 # Renaming columns and index
168 print(f"\n--- Renaming ---")
169
170 # Rename columns
171 df_renamed_cols = df.rename(columns={'Salary': 'Annual_Salary', '
172 print("After renaming columns:")
173 print(df_renamed_cols.head())
174
175 # Rename index
176 df_renamed_idx = df.rename(index={'A': 'Emp_A', 'B': 'Emp_B'})
177 print("\nAfter renaming index:")
178 print(df_renamed_idx.head())
179
180 # Rename both
181 df_renamed_both = df.rename(columns={'Salary': 'Annual_Salary'},
182 print("\nAfter renaming both columns and index:")
183 print(df_renamed_both.head())

```

7. Handling Missing Values

In [...

```
1  # Create a DataFrame with various types of missing values
2  data = {
3      'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank',
4      'Age': [25, 30, np.nan, 28, 32, 45, np.nan, 38],
5      'City': ['New York', 'Los Angeles', 'Chicago', np.nan, 'Phoen
6      'Salary': [50000, 60000, 70000, 55000, np.nan, 80000, 58000,
7      'Department': ['IT', 'HR', 'IT', 'Finance', 'IT', 'Finance',
8      'Experience': [2, 5, 8, 3, 6, 12, 4, 9],
9      'Performance': ['Good', 'Excellent', np.nan, 'Average', 'Exce
10 }
11
12 df = pd.DataFrame(data, index=['A', 'B', 'C', 'D', 'E', 'F', 'G',
13
14 print("DataFrame with missing values:")
15 print(df)
16
17 # Detecting missing values
18 print(f"\n--- Detecting Missing Values ---")
19
20 # Check for missing values
21 print("Missing values per column:")
22 print(df.isnull().sum())
23
24
25 print("\nMissing values as percentage:")
26 print((df.isnull().sum() / len(df) * 100).round(2))
27
28 print("\nAny missing values in each row:")
29 print(df.isnull().any(axis=1))
30
31 print("\nRows with any missing values:")
32 print(df[df.isnull().any(axis=1)])
33
34 # Check for non-missing values
35 print(f"\n--- Non-Missing Values ---")
36 print("Non-missing values per column:")
37 print(df.notnull().sum())
38
39
40 print("\nRows with all values present:")
41 print(df[df.notnull().all(axis=1)])
42
43 # Filling missing values
44 print(f"\n--- Filling Missing Values ---")
45
46 # Fill with constant values
47 df_fill_const = df.copy()
48 df_fill_const['Age'] = df_fill_const['Age'].fillna(30) # Fill ag
49 df_fill_const['City'] = df_fill_const['City'].fillna('Unknown')
50 print("After filling with constants:")
51 print(df_fill_const[['Name', 'Age', 'City']])
52
53 # Fill with statistical measures
54 df_fill_stats = df.copy()
55 df_fill_stats['Age'] = df_fill_stats['Age'].fillna(df_fill_stats[
56 df_fill_stats['Salary'] = df_fill_stats['Salary'].fillna(df_fill_
57 print("\nAfter filling with statistical measures:")
58
```

```

59 print(df_fill_stats[['Name', 'Age', 'Salary']])
60
61 # Fill with forward fill (ffill)
62 df_fill_forward = df.copy()
63 df_fill_forward['Performance'] = df_fill_forward['Performance'].ffill()
64 print("\nAfter forward fill for Performance:")
65 print(df_fill_forward[['Name', 'Performance']])
66
67 # Fill with backward fill (bfill)
68 df_fill_backward = df.copy()
69 df_fill_backward['Performance'] = df_fill_backward['Performance'].bfill()
70 print("\nAfter backward fill for Performance:")
71 print(df_fill_backward[['Name', 'Performance']])
72
73 # Fill with interpolate for numeric data
74 df_interpolate = df.copy()
75 df_interpolate['Age'] = df_interpolate['Age'].interpolate()
76 df_interpolate['Salary'] = df_interpolate['Salary'].interpolate()
77 print("\nAfter interpolation for numeric columns:")
78 print(df_interpolate[['Name', 'Age', 'Salary']])
79
80 # Fill with mode for categorical data
81 df_fill_mode = df.copy()
82 mode_performance = df_fill_mode['Performance'].mode()[0]
83 df_fill_mode['Performance'] = df_fill_mode['Performance'].fillna(mode_performance)
84 print(f"\nAfter filling Performance with mode ({mode_performance})")
85 print(df_fill_mode[['Name', 'Performance']])
86
87 # Using fillna() with different methods for different columns
88 df_fill_multiple = df.copy()
89 df_fill_multiple = df_fill_multiple.fillna({
90     'Age': df_fill_multiple['Age'].mean(),
91     'City': 'Unknown',
92     'Salary': df_fill_multiple['Salary'].median(),
93     'Performance': 'Good'
94 })
95 print("\nAfter filling with different methods for different columns:")
96 print(df_fill_multiple)
97
98 # Dropping missing values
99 print(f"\n--- Dropping Missing Values ---")
100
101 # Drop rows with any missing values
102 df_drop_any = df.dropna()
103 print("After dropping rows with any missing values:")
104 print(df_drop_any)
105
106 # Drop rows with all missing values
107 df_drop_all = df.copy()
108 df_drop_all.loc['I'] = [np.nan, np.nan, np.nan, np.nan, np.nan, np.nan]
109 print("\nDataFrame with a row of all NaN:")
110 print(df_drop_all)
111
112 df_drop_all_nan = df_drop_all.dropna(how='all')
113 print("\nAfter dropping rows with all missing values:")
114 print(df_drop_all_nan)
115
116 # Drop columns with any missing values
117 df_drop_cols = df.dropna(axis=1)
118 print("\nAfter dropping columns with any missing values:")

```

```

123 print(df_drop_cols)
124
125 # Drop rows with missing values in specific columns
126 df_drop_specific = df.dropna(subset=['Age', 'Salary'])
127 print("\nAfter dropping rows with missing Age or Salary:")
128 print(df_drop_specific)
129
130 # Drop rows with missing values in all specified columns
131 df_drop_all_specific = df.dropna(subset=['Age', 'Salary'], how='a')
132 print("\nAfter dropping rows with missing Age AND Salary:")
133 print(df_drop_all_specific)
134
135 # Using thresh parameter
136 df_drop_thresh = df.dropna(thresh=6) # Keep rows with at least 6
137 print("\nAfter keeping rows with at least 6 non-null values:")
138 print(df_drop_thresh)
139
140 # Working with missing values in specific contexts
141 print(f"\n--- Working with Missing Values in Context ---")
142
143 # Replace specific values with NaN
144 df_replace = df.copy()
145 df_replace['Age'] = df_replace['Age'].replace(45, np.nan) # Replace
146 print("After replacing 45 with NaN in Age:")
147 print(df_replace[['Name', 'Age']])
148
149 # Replace multiple values
150 df_replace_multiple = df.copy()
151 df_replace_multiple['Performance'] = df_replace_multiple['Performance'].replace(['Good', 'Excellent'], np.nan)
152 print("\nAfter replacing 'Good' and 'Excellent' with NaN in Performance:")
153 print(df_replace_multiple[['Name', 'Performance']])
154
155 # Check for missing values in specific data types
156 print(f"\n--- Missing Values by Data Type ---")
157 print("Missing values in numeric columns:")
158 numeric_cols = df.select_dtypes(include=[np.number]).columns
159 print(df[numeric_cols].isnull().sum())
160
161 print("\nMissing values in object columns:")
162 object_cols = df.select_dtypes(include=['object']).columns
163 print(df[object_cols].isnull().sum())
164
165 # Advanced missing value handling
166 print(f"\n--- Advanced Missing Value Handling ---")
167
168 # Create a more complex example
169 df_complex = pd.DataFrame({
170     'A': [1, 2, np.nan, 4, 5],
171     'B': [np.nan, 2, 3, np.nan, 5],
172     'C': [1, np.nan, 3, 4, np.nan],
173     'D': [1, 2, 3, 4, 5]
174 })
175
176 print("Complex DataFrame:")
177 print(df_complex)
178
179 # Fill missing values with the mean of each column
180 df_fill_mean = df_complex.fillna(df_complex.mean())
181 print("\nAfter filling with column means:")
182 print(df_fill_mean)

```

```

187
188     # Fill missing values with the median of each column
189     df_fill_median = df_complex.fillna(df_complex.median())
190     print("\nAfter filling with column medians:")
191     print(df_fill_median)
192
193     # Fill missing values with the mode of each column
194     df_fill_mode = df_complex.fillna(df_complex.mode().iloc[0])
195     print("\nAfter filling with column modes:")
196     print(df_fill_mode)
197
198     # Fill missing values with the previous value (forward fill)
199     df_fill_ffill = df_complex.fillna(method='ffill')
200     print("\nAfter forward fill:")
    print(df_fill_ffill)

    # Fill missing values with the next value (backward fill)
    df_fill_bfill = df_complex.fillna(method='bfill')
    print("\nAfter backward fill:")
    print(df_fill_bfill)

    # Fill missing values with interpolated values
    df_fill_interpolate = df_complex.interpolate()
    print("\nAfter interpolation:")
    print(df_fill_interpolate)

```

8. Data Type Conversion

In [...

```

1     # Create a DataFrame with mixed data types for conversion example
2     data = {
3         'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
4         'Age': ['25', '30', '35', '28', '32'], # String numbers
5         'Salary': [50000.0, 60000.0, 70000.0, 55000.0, 65000.0], # Float
6         'Is_Manager': [1, 0, 1, 0, 1], # Integer boolean
7         'Department': ['IT', 'HR', 'IT', 'Finance', 'IT'],
8         'Join_Date': ['2020-01-15', '2019-03-22', '2018-07-10', '2021-01-01', '2020-05-01'],
9         'Rating': ['4.5', '3.8', '4.2', '4.0', '4.7'] # String numbers
10    }
11
12    df = pd.DataFrame(data)
13
14    print("Original DataFrame with mixed data types:")
15    print(df)
16    print(f"\nData types: {df.dtypes}")
17
18    # Converting to different data types
19    print(f"\n--- Converting Data Types ---")
20
21    # Convert string numbers to numeric
22    df['Age'] = pd.to_numeric(df['Age'])
23    print("After converting Age to numeric:")
24    print(f"Age data type: {df['Age'].dtype}")
25    print(df['Age'])
26
27    # Convert with error handling
28    df['Rating'] = pd.to_numeric(df['Rating'], errors='coerce')
29    print(f"\nAfter converting Rating to numeric (with error handling)")
30

```



```

31 print(f"Rating data type: {df['Rating'].dtype}")
32 print(df['Rating'])
33
34 # Convert to integer
35 df['Age'] = df['Age'].astype(int)
36 print(f"\nAfter converting Age to integer:")
37 print(f"Age data type: {df['Age'].dtype}")
38 print(df['Age'])
39
40 # Convert to float
41 df['Rating'] = df['Rating'].astype(float)
42 print(f"\nAfter converting Rating to float:")
43 print(f"Rating data type: {df['Rating'].dtype}")
44 print(df['Rating'])
45
46 # Convert to string
47 df['Salary'] = df['Salary'].astype(str)
48 print(f"\nAfter converting Salary to string:")
49 print(f"Salary data type: {df['Salary'].dtype}")
50 print(df['Salary'])
51
52 # Convert to boolean
53 df['Is_Manager'] = df['Is_Manager'].astype(bool)
54 print(f"\nAfter converting Is_Manager to boolean:")
55 print(f"Is_Manager data type: {df['Is_Manager'].dtype}")
56 print(df['Is_Manager'])
57
58 # Convert to datetime
59 df['Join_Date'] = pd.to_datetime(df['Join_Date'])
60 print(f"\nAfter converting Join_Date to datetime:")
61 print(f"Join_Date data type: {df['Join_Date'].dtype}")
62 print(df['Join_Date'])
63
64 # Convert to category
65 df['Department'] = df['Department'].astype('category')
66 print(f"\nAfter converting Department to category:")
67 print(f"Department data type: {df['Department'].dtype}")
68 print(df['Department'])
69 print(f"Categories: {df['Department'].cat.categories}")
70
71 # Using astype() for multiple columns
72 print(f"\n--- Converting Multiple Columns ---")
73 df_multi = df.copy()
74 df_multi[['Age', 'Salary']] = df_multi[['Age', 'Salary']].astype(float)
75 print("After converting Age and Salary to float:")
76 print(df_multi[['Age', 'Salary']].dtypes)
77
78 # Using to_numeric() with different parameters
79 print(f"\n--- Using to_numeric() Parameters ---")
80
81 # Create data with some invalid values
82 data_invalid = {
83     'A': ['1', '2', 'invalid', '4', '5'],
84     'B': ['1.5', '2.7', '3.2', 'not_a_number', '5.1'],
85     'C': ['10', '20', '30', '40', '50']
86 }
87 df_invalid = pd.DataFrame(data_invalid)
88
89 print("DataFrame with invalid numeric values:")
90 print(df_invalid)
91

```

```

95
96 # Convert with errors='coerce' (invalid becomes NaN)
97 df_coerce = df_invalid.copy()
98 df_coerce['A'] = pd.to_numeric(df_coerce['A'], errors='coerce')
99 df_coerce['B'] = pd.to_numeric(df_coerce['B'], errors='coerce')
100 print("\nAfter converting with errors='coerce':")
101 print(df_coerce)
102
103 # Convert with errors='ignore' (keeps as string)
104 df_ignore = df_invalid.copy()
105 df_ignore['A'] = pd.to_numeric(df_ignore['A'], errors='ignore')
106 print("\nAfter converting with errors='ignore':")
107 print(df_ignore['A'])
108
109 # Convert with downcast parameter
110 df_downcast = df_invalid.copy()
111 df_downcast['C'] = pd.to_numeric(df_downcast['C'], downcast='integer')
112 print(f"\nAfter converting C with downcast='integer':")
113 print(f"C data type: {df_downcast['C'].dtype}")
114
115 # Using infer_objects() to automatically infer types
116 print(f"\n--- Using infer_objects() ---")
117 df_infer = df.copy()
118 df_infer['Age'] = df_infer['Age'].astype(str) # Convert back to
119 df_infer['Salary'] = df_infer['Salary'].astype(str) # Convert back to
120 print("DataFrame with string numbers:")
121 print(df_infer[['Age', 'Salary']].dtypes)
122
123 df_infer = df_infer.infer_objects()
124 print("\nAfter using infer_objects():")
125 print(df_infer[['Age', 'Salary']].dtypes)
126
127 # Converting datetime with different formats
128 print(f"\n--- DateTime Conversion ---")
129 date_data = {
130     'Date1': ['2023-01-15', '2023-02-20', '2023-03-25'],
131     'Date2': ['15/01/2023', '20/02/2023', '25/03/2023'],
132     'Date3': ['Jan 15, 2023', 'Feb 20, 2023', 'Mar 25, 2023'],
133     'DateTime': ['2023-01-15 10:30:00', '2023-02-20 14:45:00', '2023-03-25 09:00:00']
134 }
135 df_dates = pd.DataFrame(date_data)
136
137 print("DataFrame with different date formats:")
138 print(df_dates)
139
140 # Convert with default format
141 df_dates['Date1'] = pd.to_datetime(df_dates['Date1'])
142 print(f"\nDate1 after conversion: {df_dates['Date1'].dtype}")
143
144 # Convert with specific format
145 df_dates['Date2'] = pd.to_datetime(df_dates['Date2'], format='%d/%m/%Y')
146 print(f"Date2 after conversion: {df_dates['Date2'].dtype}")
147
148 # Convert with infer_datetime_format
149 df_dates['Date3'] = pd.to_datetime(df_dates['Date3'], infer_datetime_format=True)
150 print(f"Date3 after conversion: {df_dates['Date3'].dtype}")
151
152 # Convert datetime string
153 df_dates['DateTime'] = pd.to_datetime(df_dates['DateTime'])
154 print(f"DateTime after conversion: {df_dates['DateTime'].dtype}")
155

```

```

159
160     print("\nConverted dates:")
161     print(df_dates)
162
163     # Converting to categorical with custom categories
164     print(f"\n--- Categorical Conversion ---")
165     df_cat = df.copy()
166     df_cat['Department'] = pd.Categorical(df_cat['Department'],
167                                         categories=['HR', 'IT', 'Finance'],
168                                         ordered=True)
169
170     print("Department as ordered categorical:")
171     print(df_cat['Department'])
172     print(f"Categories: {df_cat['Department'].cat.categories}")
173     print(f"Is ordered: {df_cat['Department'].cat.ordered}")
174
175     # Converting boolean with different representations
176     print(f"\n--- Boolean Conversion ---")
177     bool_data = {
178         'Yes_No': ['Yes', 'No', 'Yes', 'No', 'Yes'],
179         'True_False': ['True', 'False', 'True', 'False', 'True'],
180         'Numbers': [1, 0, 1, 0, 1],
181         'Strings': ['true', 'false', 'true', 'false', 'true']
182     }
183     df_bool = pd.DataFrame(bool_data)
184
185     print("DataFrame with different boolean representations:")
186     print(df_bool)
187
188     # Convert different boolean representations
189     df_bool['Yes_No'] = df_bool['Yes_No'].map({'Yes': True, 'No': False})
190     df_bool['True_False'] = df_bool['True_False'].map({'True': True, 'False': False})
191     df_bool['Numbers'] = df_bool['Numbers'].astype(bool)
192     df_bool['Strings'] = df_bool['Strings'].map({'true': True, 'false': False})
193
194     print("\nAfter converting to boolean:")
195     print(df_bool)
196     print(f"\nData types:\n{df_bool.dtypes}")
197
198     # Using convert_dtypes() for optimal data types
199     print(f"\n--- Using convert_dtypes() ---")
200     df_convert = df.copy()
201     df_convert['Age'] = df_convert['Age'].astype(str)
202     df_convert['Salary'] = df_convert['Salary'].astype(str)
203     df_convert['Is_Manager'] = df_convert['Is_Manager'].astype(str)
204
205     print("DataFrame with suboptimal data types:")
206     print(df_convert.dtypes)
207
208     df_convert = df_convert.convert_dtypes()
209     print("\nAfter using convert_dtypes():")
210     print(df_convert.dtypes)

```

9. Sorting and Ranking

In [...

```

1     # Create a comprehensive DataFrame for sorting and ranking example
2     data = {

```

```

3      'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank',
4      'Age': [25, 30, 35, 28, 32, 45, 29, 38],
5      'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Ph
6      'Salary': [50000, 60000, 70000, 55000, 65000, 80000, 58000, 7
7      'Department': ['IT', 'HR', 'IT', 'Finance', 'IT', 'Finance',
8      'Experience': [2, 5, 8, 3, 6, 12, 4, 9],
9      'Performance': ['Good', 'Excellent', 'Good', 'Average', 'Exce
10 }
11 df = pd.DataFrame(data, index=['A', 'B', 'C', 'D', 'E', 'F', 'G',
12
13 print("Original DataFrame:")
14 print(df)
15
16 # Basic sorting
17 print(f"\n--- Basic Sorting ---")
18
19 # Sort by a single column (ascending)
20 df_sorted_age = df.sort_values('Age')
21 print("Sorted by Age (ascending):")
22 print(df_sorted_age)
23
24 # Sort by a single column (descending)
25 df_sorted_salary = df.sort_values('Salary', ascending=False)
26 print("\nSorted by Salary (descending):")
27 print(df_sorted_salary)
28
29 # Sort by multiple columns
30 df_sorted_multi = df.sort_values(['Department', 'Salary'], ascend
31 print("\nSorted by Department (ascending) then Salary (descending
32 print(df_sorted_multi)
33
34 # Sort by index
35 df_sorted_index = df.sort_index()
36 print("\nSorted by index:")
37 print(df_sorted_index)
38
39 # Sort by index in descending order
40 df_sorted_index_desc = df.sort_index(ascending=False)
41 print("\nSorted by index (descending):")
42 print(df_sorted_index_desc)
43
44 # Sorting with missing values
45 print(f"\n--- Sorting with Missing Values ---")
46
47 # Create DataFrame with missing values
48 df_with_nulls = df.copy()
49 df_with_nulls.loc['A', 'Salary'] = np.nan
50 df_with_nulls.loc['C', 'Age'] = np.nan
51
52 print("DataFrame with missing values:")
53 print(df_with_nulls[['Name', 'Age', 'Salary']])
54
55 # Sort with na_position='first' (missing values first)
56 df_nulls_first = df_with_nulls.sort_values('Salary', na_position=
57 print("\nSorted by Salary with missing values first:")
58 print(df_nulls_first[['Name', 'Age', 'Salary']])
59
60 # Sort with na_position='last' (missing values last)
61 df_nulls_last = df_with_nulls.sort_values('Age', na_position='las
62 print("\nSorted by Age with missing values last:")

```

```

67     print(df_nulls_last[['Name', 'Age', 'Salary']])
68
69     # Custom sorting
70     print(f"\n--- Custom Sorting ---")
71
72     # Sort by custom order for categorical data
73     performance_order = ['Excellent', 'Good', 'Average']
74     df_custom = df.copy()
75     df_custom['Performance'] = pd.Categorical(df_custom['Performance'],
76     df_custom_sorted = df_custom.sort_values('Performance')
77     print("Sorted by Performance (custom order):")
78     print(df_custom_sorted[['Name', 'Performance']])
79
80
81     # Sort by string length
82     df['Name_Length'] = df['Name'].str.len()
83     df_sorted_length = df.sort_values('Name_Length')
84     print("\nSorted by Name length:")
85     print(df_sorted_length[['Name', 'Name_Length']])
86
87     # Ranking
88     print(f"\n--- Ranking ---")
89
90     # Basic ranking
91     df['Salary_Rank'] = df['Salary'].rank()
92     print("DataFrame with Salary ranking:")
93     print(df[['Name', 'Salary', 'Salary_Rank']])
94
95     # Ranking with different methods
96     df['Salary_Rank_Avg'] = df['Salary'].rank(method='average')
97     df['Salary_Rank_Min'] = df['Salary'].rank(method='min')
98     df['Salary_Rank_Max'] = df['Salary'].rank(method='max')
99     df['Salary_Rank_Dense'] = df['Salary'].rank(method='dense')
100
101
102     print("\nDifferent ranking methods for Salary:")
103     print(df[['Name', 'Salary', 'Salary_Rank_Avg', 'Salary_Rank_Min',
104
105     # Ranking in descending order
106     df['Salary_Rank_Desc'] = df['Salary'].rank(ascending=False)
107     print("\nSalary ranking (descending):")
108     print(df[['Name', 'Salary', 'Salary_Rank_Desc']])
109
110     # Ranking with ties
111     print(f"\n--- Ranking with Ties ---")
112
113     # Create data with ties
114     data_ties = {
115         'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
116         'Score': [85, 90, 85, 90, 88]
117     }
118     df_ties = pd.DataFrame(data_ties)
119
120
121     print("DataFrame with ties:")
122     print(df_ties)
123
124     # Different methods for handling ties
125     df_ties['Rank_Average'] = df_ties['Score'].rank(method='average')
126     df_ties['Rank_Min'] = df_ties['Score'].rank(method='min')
127     df_ties['Rank_Max'] = df_ties['Score'].rank(method='max')
128     df_ties['Rank_Dense'] = df_ties['Score'].rank(method='dense')
129
130

```

```

131 print("\nDifferent tie-breaking methods:")
132 print(df_ties)
133
134 # Percentile ranking
135 print(f"\n--- Percentile Ranking ---")
136 df['Salary_Percentile'] = df['Salary'].rank(pct=True)
137 print("Salary percentile ranking:")
138 print(df[['Name', 'Salary', 'Salary_Percentile']])
139
140 # Ranking by groups
141 print(f"\n--- Ranking by Groups ---")
142 df['Salary_Rank_by_Dept'] = df.groupby('Department')['Salary'].rank()
143 print("Salary ranking within each department:")
144 print(df[['Name', 'Department', 'Salary', 'Salary_Rank_by_Dept']])
145
146 # Advanced sorting examples
147 print(f"\n--- Advanced Sorting Examples ---")
148
149 # Sort by multiple criteria with different orders
150 df_advanced = df.sort_values(['Department', 'Experience', 'Salary'],
151                               ascending=[True, False, True])
152 print("Sorted by Department (asc), Experience (desc), Salary (asc)")
153 print(df_advanced[['Name', 'Department', 'Experience', 'Salary']])
154
155 # Sort by calculated column
156 df['Salary_Per_Year'] = df['Salary'] / df['Experience']
157 df_sorted_ratio = df.sort_values('Salary_Per_Year', ascending=False)
158 print("\nSorted by Salary per year of experience:")
159 print(df_sorted_ratio[['Name', 'Salary', 'Experience', 'Salary_Per_Year']])
160
161 # Sort by string operations
162 df['City_Length'] = df['City'].str.len()
163 df_sorted_city = df.sort_values('City_Length')
164 print("\nSorted by city name length:")
165 print(df_sorted_city[['Name', 'City', 'City_Length']])
166
167 # Sort by multiple string criteria
168 df_sorted_strings = df.sort_values(['Department', 'Name'])
169 print("\nSorted by Department then Name:")
170 print(df_sorted_strings[['Name', 'Department']])
171
172 # Using nlargest and nsmallest
173 print(f"\n--- Using nlargest and nsmallest ---")
174
175 # Get top 3 highest salaries
176 top_3_salaries = df.nlargest(3, 'Salary')
177 print("Top 3 highest salaries:")
178 print(top_3_salaries[['Name', 'Salary']])
179
180 # Get bottom 3 lowest salaries
181 bottom_3_salaries = df.nsmallest(3, 'Salary')
182 print("\nBottom 3 lowest salaries:")
183 print(bottom_3_salaries[['Name', 'Salary']])
184
185 # Get top 2 from each department
186 top_by_dept = df.groupby('Department').apply(lambda x: x.nlargest(2, 'Salary'))
187 print("\nTop 2 salaries from each department:")
188 print(top_by_dept[['Name', 'Department', 'Salary']])
189
190 # Sorting with key functions
191

```

```

195 print(f"\n--- Sorting with Key Functions ---")
196
197 # Sort by absolute difference from mean salary
mean_salary = df['Salary'].mean()
df['Salary_Diff_From_Mean'] = abs(df['Salary'] - mean_salary)
df_sorted_diff = df.sort_values('Salary_Diff_From_Mean')
print("Sorted by absolute difference from mean salary:")
print(df_sorted_diff[['Name', 'Salary', 'Salary_Diff_From_Mean']])

# Sort by custom function (e.g., by last letter of name)
df['Name_Last_Letter'] = df['Name'].str[-1]
df_sorted_last_letter = df.sort_values('Name_Last_Letter')
print("\nSorted by last letter of name:")
print(df_sorted_last_letter[['Name', 'Name_Last_Letter']])

```

10. Basic Aggregation Functions

In [...

```

1  # Create a comprehensive DataFrame for aggregation examples
2  data = {
3      'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank',
4      'Age': [25, 30, 35, 28, 32, 45, 29, 38, 26, 42],
5      'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Ph
6      'Salary': [50000, 60000, 70000, 55000, 65000, 80000, 58000, 7
7      'Department': ['IT', 'HR', 'IT', 'Finance', 'IT', 'Finance',
8      'Experience': [2, 5, 8, 3, 6, 12, 4, 9, 1, 10],
9      'Performance': ['Good', 'Excellent', 'Good', 'Average', 'Exce
10     'Bonus': [5000, 8000, 7000, 3000, 9000, 12000, 4000, 10000, 2
11
12 }
13 df = pd.DataFrame(data, index=['A', 'B', 'C', 'D', 'E', 'F', 'G',
14
15 print("Original DataFrame:")
16 print(df)
17
18 # Basic aggregation functions
19 print(f"\n--- Basic Aggregation Functions ---")
20
21 # Sum
22 print("Sum of numeric columns:")
23 print(df[['Age', 'Salary', 'Experience', 'Bonus']].sum())
24
25 # Mean
26 print("\nMean of numeric columns:")
27 print(df[['Age', 'Salary', 'Experience', 'Bonus']].mean())
28
29 # Median
30 print("\nMedian of numeric columns:")
31 print(df[['Age', 'Salary', 'Experience', 'Bonus']].median())
32
33 # Mode
34 print("\nMode of categorical columns:")
35 print(df[['Department', 'Performance']].mode())
36
37 # Standard deviation
38 print("\nStandard deviation of numeric columns:")
39 print(df[['Age', 'Salary', 'Experience', 'Bonus']].std())
40
41
42

```



```

43 # Variance
44 print("\nVariance of numeric columns:")
45 print(df[['Age', 'Salary', 'Experience', 'Bonus']].var())
46
47 # Min and Max
48 print("\nMinimum values:")
49 print(df[['Age', 'Salary', 'Experience', 'Bonus']].min())
50
51 print("\nMaximum values:")
52 print(df[['Age', 'Salary', 'Experience', 'Bonus']].max())
53
54 # Count
55 print("\nCount of non-null values:")
56 print(df.count())
57
58 # Count unique values
59 print("\nCount of unique values:")
60 print(df.nunique())
61
62 # Describe method
63 print(f"\n--- Describe Method ---")
64 print("Statistical summary of numeric columns:")
65 print(df.describe())
66
67 print("\nStatistical summary including all columns:")
68 print(df.describe(include='all'))
69
70 # Aggregation with groupby
71 print(f"\n--- Aggregation with GroupBy ---")
72
73 # Group by Department
74 dept_stats = df.groupby('Department').agg({
75     'Salary': ['mean', 'median', 'std', 'count'],
76     'Age': ['mean', 'min', 'max'],
77     'Experience': ['mean', 'sum']
78 })
79 print("Statistics by Department:")
80 print(dept_stats)
81
82 # Flatten column names
83 dept_stats.columns = ['_'.join(col).strip() for col in dept_stats
84     .columns.values]
85 print("\nFlattened column names:")
86 print(dept_stats)
87
88 # Multiple aggregation functions on same column
89 print(f"\n--- Multiple Aggregations on Same Column ---")
90 salary_stats = df.groupby('Department')['Salary'].agg(['count', '
91 print("Salary statistics by Department:")
92 print(salary_stats)
93
94 # Custom aggregation functions
95 print(f"\n--- Custom Aggregation Functions ---")
96
97 # Define custom functions
98 def range_func(x):
99     return x.max() - x.min()
100
101 def iqr_func(x):
102     return x.quantile(0.75) - x.quantile(0.25)
103
104
105
106

```



```

107 # Apply custom functions
108 custom_stats = df.groupby('Department')['Salary'].agg(['mean', 's
109 custom_stats.columns = ['Mean', 'Std', 'Range', 'IQR']
110 print("Custom salary statistics by Department:")
111 print(custom_stats)
112
113 # Aggregation with multiple columns
114 print(f"\n--- Aggregation with Multiple Columns ---")
115 multi_col_stats = df.groupby('Department')[['Salary', 'Age', 'Exp
116 print("Multiple column statistics by Department:")
117 print(multi_col_stats)
118
119 # Using apply with custom functions
120 print(f"\n--- Using Apply with Custom Functions ---")
121
122 # Custom function for salary analysis
123 def salary_analysis(group):
124     return pd.Series({
125         'avg_salary': group['Salary'].mean(),
126         'max_salary': group['Salary'].max(),
127         'min_salary': group['Salary'].min(),
128         'salary_range': group['Salary'].max() - group['Salary'].m
129         'employee_count': len(group)
130     })
131
132 salary_analysis_result = df.groupby('Department').apply(salary_an
133 print("Custom salary analysis by Department:")
134 print(salary_analysis_result)
135
136 # Aggregation with filtering
137 print(f"\n--- Aggregation with Filtering ---")
138
139 # Filter data before aggregation
140 high_performers = df[df['Performance'].isin(['Excellent', 'Good'])
141 high_perf_stats = high_performers.groupby('Department')['Salary']
142 print("Salary statistics for high performers by Department:")
143 print(high_perf_stats)
144
145 # Conditional aggregation
146 print(f"\n--- Conditional Aggregation ---")
147
148 # Count by condition
149 dept_high_salary = df.groupby('Department').apply(
150     lambda x: (x['Salary'] > 60000).sum()
151 )
152 print("Count of employees with salary > 60000 by Department:")
153 print(dept_high_salary)
154
155 # Percentage by condition
156 dept_high_salary_pct = df.groupby('Department').apply(
157     lambda x: (x['Salary'] > 60000).mean() * 100
158 )
159 print("\nPercentage of employees with salary > 60000 by Departmen
160 print(dept_high_salary_pct)
161
162 # Aggregation with multiple groupby columns
163 print(f"\n--- Multiple GroupBy Columns ---")
164
165 # Group by Department and Performance
166 dept_perf_stats = df.groupby(['Department', 'Performance'])['Sala
167

```

```

171 print("Salary statistics by Department and Performance:")
172 print(dept_perf_stats)
173
174 # Pivot table style aggregation
175 print(f"\n--- Pivot Table Style Aggregation ---")
176 pivot_stats = df.pivot_table(
177     values='Salary',
178     index='Department',
179     columns='Performance',
180     aggfunc=['mean', 'count'],
181     fill_value=0
182 )
183
184 print("Pivot table of Salary by Department and Performance:")
185 print(pivot_stats)
186
187 # Cross-tabulation
188 print(f"\n--- Cross-tabulation ---")
189 crosstab = pd.crosstab(df['Department'], df['Performance'])
190 print("Cross-tabulation of Department and Performance:")
191 print(crosstab)
192
193 # Aggregation with time series (if applicable)
194 print(f"\n--- Time Series Aggregation ---")
195
196 # Create a time series example
197 dates = pd.date_range('2023-01-01', periods=10, freq='D')
198 df_time = df.copy()
199 df_time['Date'] = dates
200 df_time['Daily_Sales'] = np.random.randint(1000, 5000, 10)
201
202
203 print("Time series DataFrame:")
204 print(df_time[['Name', 'Date', 'Daily_Sales']].head())
205
206 # Resample by week
207 weekly_sales = df_time.set_index('Date')['Daily_Sales'].resample('W').sum()
208 print("\nWeekly sales totals:")
209 print(weekly_sales)
210
211 # Rolling window aggregation
212 print(f"\n--- Rolling Window Aggregation ---")
213 df_time_sorted = df_time.sort_values('Date')
214 df_time_sorted['Rolling_Avg'] = df_time_sorted['Daily_Sales'].rolling(3).mean()
215 print("Daily sales with 3-day rolling average:")
216 print(df_time_sorted[['Date', 'Daily_Sales', 'Rolling_Avg']].head())
217
218 # Expanding window aggregation
219 print(f"\n--- Expanding Window Aggregation ---")
220 df_time_sorted['Expanding_Sum'] = df_time_sorted['Daily_Sales'].rolling('all').sum()
221 print("Daily sales with expanding sum:")
222 print(df_time_sorted[['Date', 'Daily_Sales', 'Expanding_Sum']].head())
223
224 # Aggregation with missing values
225 print(f"\n--- Aggregation with Missing Values ---")
226
227 # Create DataFrame with missing values
228 df_missing = df.copy()
229 df_missing.loc['A', 'Salary'] = np.nan
230 df_missing.loc['C', 'Age'] = np.nan
231
232
233 print("DataFrame with missing values:")
234

```

```

235     print(df_missing[['Name', 'Age', 'Salary']].head())
236
237     # Aggregation with skipna=True (default)
238     print("\nMean with skipna=True (default):")
239     print(df_missing[['Age', 'Salary']].mean())
240
241     # Aggregation with skipna=False
242     print("\nMean with skipna=False:")
243     print(df_missing[['Age', 'Salary']].mean(skipna=False))
244
245     # Count non-null values
246     print("\nCount of non-null values:")
247     print(df_missing[['Age', 'Salary']].count())
248
249     # Advanced aggregation examples
250     print(f"\n--- Advanced Aggregation Examples ---")
251
252     # Weighted average
253     df['Weight'] = np.random.rand(10)
254     weighted_avg = (df['Salary'] * df['Weight']).sum() / df['Weight']
255     print(f"Weighted average salary: {weighted_avg:.2f}")
256
257     # Percentile aggregation
258     percentiles = df.groupby('Department')['Salary'].quantile([0.25,
259     print("\nSalary percentiles by Department:")
260     print(percentiles)
261
262     # Aggregation with custom grouping
263     print(f"\n--- Custom Grouping ---")
264
265     # Create age groups
266     df['Age_Group'] = pd.cut(df['Age'], bins=[0, 30, 40, 100], labels
267     age_group_stats = df.groupby('Age_Group')['Salary'].agg(['mean',
268     print("Salary statistics by Age Group:")
269     print(age_group_stats)

```

11. DataFrame Operations and Arithmetic

In [...

```

1     # Create DataFrames for arithmetic operations
2     data1 = {
3         'A': [1, 2, 3, 4, 5],
4         'B': [10, 20, 30, 40, 50],
5         'C': [100, 200, 300, 400, 500]
6     }
7
8     df1 = pd.DataFrame(data1, index=['X', 'Y', 'Z', 'W', 'V'])
9
10    data2 = {
11        'A': [2, 4, 6, 8, 10],
12        'B': [5, 10, 15, 20, 25],
13        'C': [50, 100, 150, 200, 250]
14    }
15    df2 = pd.DataFrame(data2, index=['X', 'Y', 'Z', 'W', 'V'])
16
17    print("DataFrame 1:")
18    print(df1)
19    print("\nDataFrame 2:")

```

```

20     print(df2)
21
22     # Basic arithmetic operations
23     print(f"\n--- Basic Arithmetic Operations ---")
24
25     # Addition
26     print("Addition (df1 + df2):")
27     print(df1 + df2)
28
29     # Subtraction
30     print("\nSubtraction (df1 - df2):")
31     print(df1 - df2)
32
33     # Multiplication
34     print("\nMultiplication (df1 * df2):")
35     print(df1 * df2)
36
37     # Division
38     print("\nDivision (df1 / df2):")
39     print(df1 / df2)
40
41     # Floor division
42     print("\nFloor division (df1 // df2):")
43     print(df1 // df2)
44
45     # Modulo
46     print("\nModulo (df1 % df2):")
47     print(df1 % df2)
48
49     # Power
50     print("\nPower (df1 ** 2):")
51     print(df1 ** 2)
52
53     # Arithmetic with scalars
54     print(f"\n--- Arithmetic with Scalars ---")
55
56     # Add scalar to all elements
57     print("Add 10 to all elements:")
58     print(df1 + 10)
59
60     # Multiply by scalar
61     print("\nMultiply by 2:")
62     print(df1 * 2)
63
64     # Divide by scalar
65     print("\nDivide by 5:")
66     print(df1 / 5)
67
68     # Arithmetic with Series
69     print(f"\n--- Arithmetic with Series ---")
70
71     # Create a Series
72     series1 = pd.Series([1, 2, 3, 4, 5], index=['X', 'Y', 'Z', 'W', 'V'])
73     print("Series:")
74     print(series1)
75
76     # Add Series to DataFrame
77     print("\nDataFrame + Series:")
78     print(df1 + series1)
79
80
81
82
83

```

```

84     # Multiply DataFrame by Series
85     print("\nDataFrame * Series:")
86     print(df1 * series1)
87
88     # Broadcasting with different indices
89     print(f"\n--- Broadcasting with Different Indices ---")
90
91     # Series with different index
92     series2 = pd.Series([10, 20], index=['X', 'Y'])
93     print("Series with different index:")
94     print(series2)
95
96     # Broadcasting
97     print("\nDataFrame + Series (broadcasting):")
98     print(df1 + series2)
99
100
101     # Arithmetic with mismatched DataFrames
102     print(f"\n--- Arithmetic with Mismatched DataFrames ---")
103
104     # Create DataFrame with different columns
105     df3 = pd.DataFrame({
106         'A': [1, 2, 3],
107         'D': [100, 200, 300]
108     }, index=['X', 'Y', 'Z'])
109
110     print("DataFrame 3:")
111     print(df3)
112
113     # Addition with mismatched columns
114     print("\nAddition with mismatched columns:")
115     print(df1 + df3)
116
117     # Arithmetic with different shapes
118     print(f"\n--- Arithmetic with Different Shapes ---")
119
120
121     # Create smaller DataFrame
122     df4 = pd.DataFrame({
123         'A': [1, 2],
124         'B': [10, 20]
125     }, index=['X', 'Y'])
126
127     print("Smaller DataFrame:")
128     print(df4)
129
130     # Addition with different shapes
131     print("\nAddition with different shapes:")
132     print(df1 + df4)
133
134     # Using add(), sub(), mul(), div() methods
135     print(f"\n--- Using Arithmetic Methods ---")
136
137     # Add method
138     print("Using add() method:")
139     print(df1.add(df2))
140
141     # Subtract method
142     print("\nUsing sub() method:")
143     print(df1.sub(df2))
144
145     # Multiply method
146
147

```

```

148 print("\nUsing mul() method:")
149 print(df1.mul(df2))
150
151 # Divide method
152 print("\nUsing div() method:")
153 print(df1.div(df2))
154
155 # Arithmetic methods with fill_value
156 print(f"\n--- Arithmetic Methods with fill_value ---")
157
158 # Create DataFrames with missing values
159 df5 = pd.DataFrame({
160     'A': [1, 2, np.nan, 4],
161     'B': [10, np.nan, 30, 40]
162 }, index=['X', 'Y', 'Z', 'W'])
163
164
165 df6 = pd.DataFrame({
166     'A': [2, 4, 6, 8],
167     'B': [5, 10, 15, 20]
168 }, index=['X', 'Y', 'Z', 'W'])
169
170 print("DataFrame with missing values:")
171 print(df5)
172 print("\nDataFrame without missing values:")
173 print(df6)
174
175 # Addition with fill_value
176 print("\nAddition with fill_value=0:")
177 print(df5.add(df6, fill_value=0))
178
179 # Comparison operations
180 print(f"\n--- Comparison Operations ---")
181
182 # Greater than
183 print("df1 > df2:")
184 print(df1 > df2)
185
186 # Less than or equal
187 print("\ndf1 <= df2:")
188 print(df1 <= df2)
189
190 # Equal to
191 print("\ndf1 == df2:")
192 print(df1 == df2)
193
194 # Not equal to
195 print("\ndf1 != df2:")
196 print(df1 != df2)
197
198 # Comparison with scalars
199 print(f"\n--- Comparison with Scalars ---")
200
201 # Greater than scalar
202 print("df1 > 3:")
203 print(df1 > 3)
204
205 # Between values
206 print("\nValues between 2 and 4:")
207 print((df1 > 2) & (df1 < 4))
208
209
210
211

```

```

212 # Logical operations
213 print(f"\n--- Logical Operations ---")
214
215 # AND operation
216 print("df1 > 2 AND df1 < 4:")
217 print((df1 > 2) & (df1 < 4))
218
219 # OR operation
220 print("\ndf1 < 2 OR df1 > 4:")
221 print((df1 < 2) | (df1 > 4))
222
223 # NOT operation
224 print("\nNOT (df1 > 3):")
225 print(~(df1 > 3))
226
227
228 # String operations
229 print(f"\n--- String Operations ---")
230
231 # Create DataFrame with string data
232 str_data = {
233     'Name': ['Alice', 'Bob', 'Charlie', 'Diana'],
234     'City': ['New York', 'Los Angeles', 'Chicago', 'Houston'],
235     'Department': ['IT', 'HR', 'IT', 'Finance']
236 }
237 df_str = pd.DataFrame(str_data)
238
239 print("String DataFrame:")
240 print(df_str)
241
242 # String concatenation
243 df_str['Full_Info'] = df_str['Name'] + ' from ' + df_str['City']
244 print("\nAfter string concatenation:")
245 print(df_str['Full_Info'])
246
247 # String methods
248 print("\nString methods:")
249 print("Names in uppercase:", df_str['Name'].str.upper().tolist())
250 print("Cities in lowercase:", df_str['City'].str.lower().tolist())
251 print("Name lengths:", df_str['Name'].str.len().tolist())
252
253 # Mathematical functions
254 print(f"\n--- Mathematical Functions ---")
255
256 # Absolute value
257 print("Absolute value of df1 - df2:")
258 print(np.abs(df1 - df2))
259
260 # Square root
261 print("\nSquare root of df1:")
262 print(np.sqrt(df1))
263
264 # Logarithm
265 print("\nNatural logarithm of df1:")
266 print(np.log(df1))
267
268 # Trigonometric functions
269 print("\nSine of df1 (in radians):")
270 print(np.sin(df1))
271
272 # Statistical functions

```

```

276     print(f"\n--- Statistical Functions ---")
277
278     # Mean across rows
279     print("Mean across rows:")
280     print(df1.mean(axis=1))
281
282     # Sum across columns
283     print("\nSum across columns:")
284     print(df1.sum(axis=0))
285
286     # Cumulative operations
287     print(f"\n--- Cumulative Operations ---")
288
289     # Cumulative sum
290     print("Cumulative sum:")
291     print(df1.cumsum())
292
293     # Cumulative product
294     print("\nCumulative product:")
295     print(df1.cumprod())
296
297     # Cumulative maximum
298     print("\nCumulative maximum:")
299     print(df1.cummax())
300
301     # Cumulative minimum
302     print("\nCumulative minimum:")
303     print(df1.cummin())
304
305     # Window functions
306     print(f"\n--- Window Functions ---")
307
308     # Rolling window
309     print("3-period rolling mean:")
310     print(df1.rolling(window=3).mean())
311
312     # Expanding window
313     print("\nExpanding mean:")
314     print(df1.expanding().mean())
315
316     # Exponential weighted functions
317     print("\nExponential weighted mean (alpha=0.5):")
318     print(df1.ewm(alpha=0.5).mean())
319
320     # Custom operations
321     print(f"\n--- Custom Operations ---")
322
323     # Apply custom function
324     def custom_func(x):
325         return x * 2 + 1
326
327     print("Custom function (x * 2 + 1):")
328     print(df1.apply(custom_func))
329
330     # Apply function to each column
331     print("\nApply function to each column:")
332     print(df1.apply(lambda x: x.max() - x.min()))
333
334     # Apply function to each row
335     print("\nApply function to each row:")

```



```

340     print(df1.apply(lambda x: x.max() - x.min(), axis=1))
341
342     # Element-wise operations with applymap
343     print("\nElement-wise operations with applymap:")
344     print(df1.applymap(lambda x: x ** 2))
345
346     # Vectorized operations
347     print(f"\n--- Vectorized Operations ---")
348
349     # Create a more complex example
350     df_complex = pd.DataFrame({
351         'Sales': [1000, 1500, 2000, 1200, 1800],
352         'Cost': [600, 900, 1200, 720, 1080],
353         'Quantity': [10, 15, 20, 12, 18]
354     }, index=['Jan', 'Feb', 'Mar', 'Apr', 'May'])
355
356
357     print("Complex DataFrame:")
358     print(df_complex)
359
360     # Calculate profit margin
361     df_complex['Profit'] = df_complex['Sales'] - df_complex['Cost']
362     df_complex['Profit_Margin'] = (df_complex['Profit'] / df_complex[
363     df_complex['Unit_Price'] = df_complex['Sales'] / df_complex['Quan
364
365
366     print("\nAfter calculations:")
367     print(df_complex)
368
369     # Conditional operations
370     print(f"\n--- Conditional Operations ---")
371
372     # Where condition
373     print("Where Sales > 1500, otherwise 0:")
374     print(df_complex['Sales'].where(df_complex['Sales'] > 1500, 0))
375
376     # Numpy where
377     print("\nUsing np.where (Sales > 1500: 'High', else: 'Low'):")
378     print(np.where(df_complex['Sales'] > 1500, 'High', 'Low'))
379
380     # Clip values
381     print("\nClipping values between 1000 and 2000:")
382     print(df_complex['Sales'].clip(1000, 2000))
383
384     # Round values
385     print("\nRounding to nearest 100:")
386     print(df_complex['Sales'].round(-2))

```

Summary

This module has covered all the essential topics for working with Pandas DataFrames:

Key Topics Covered:

1. **Introduction to DataFrames** - Understanding the structure and basic properties
2. **Creating DataFrames** - From various sources (dictionaries, lists, arrays, etc.)
3. **DataFrame Attributes and Methods** - Properties, information, and basic operations

4. **Indexing and Selection** - Using loc, iloc, at, iat for data access
5. **Filtering and Boolean Indexing** - Advanced filtering techniques and conditions
6. **Adding and Removing Columns/Rows** - Data manipulation and restructuring
7. **Handling Missing Values** - Detection, filling, and dropping missing data
8. **Data Type Conversion** - Converting between different data types
9. **Sorting and Ranking** - Organizing data and creating rankings
10. **Basic Aggregation Functions** - Statistical operations and grouping
11. **DataFrame Operations and Arithmetic** - Mathematical operations and comparisons

Key Skills Developed:

- Creating and manipulating DataFrames
- Selecting and filtering data effectively
- Cleaning and preprocessing data
- Performing statistical analysis
- Working with missing values
- Converting data types appropriately
- Sorting and ranking data
- Performing aggregation operations
- Applying mathematical operations

Next Steps:

- Practice with real-world datasets
- Explore advanced pandas features
- Learn about data visualization (Module 7)
- Apply these skills to machine learning projects (Modules 8-9)

In [...

1	
2	